

Prácticas de Programación en C

Estructuras de Datos, Ordenamiento Burbuja,
Búsqueda Binaria y CRUD

1. Inventario con Ordenamiento Burbuja y Búsqueda Binaria (valor fijo)

Array de 20 productos con struct (nroProducto, nombre, precio). Muestra el inventario original, ordena por número de producto con burbuja, realiza búsqueda binaria con valor fijo (hardcodeado) y muestra el inventario final ordenado.

```
#include <stdio.h>

/* Definir la estructura de datos */
struct Producto {
    int nroProducto;
    char nombre[20];
    float precio;
};

int main() {
    /* Inicializar los datos */
    struct Producto inventario[] = {
        {58293012, "Monitor", 120.00},
        {19485726, "Mousepad", 10.00},
        {83746501, "Procesador", 200.00},
        {37482910, "Teclado", 25.50},
        {92837465, "Fuente ATX", 75.00},
        {48291037, "Webcam", 55.00},
        {10293847, "Silla Gamer", 250.00},
        {74659283, "Impresora", 85.00},
        {29384756, "Disco SSD", 95.00},
        {65748392, "Mouse", 15.75},
```

```

        {51928374, "Parlantes", 40.20},
        {84756291, "Cables HDMI", 8.50},
        {38475629, "Auriculares", 65.50},
        {91028374, "Gabinete", 60.00},
        {27384950, "Microfono", 30.00},
        {46573829, "Adaptador USB", 5.99},
        {15263748, "Memoria RAM", 50.00},
        {73849506, "Pendrive", 12.99},
        {62738495, "Placa Video", 350.00},
        {39485761, "Escritorio", 150.00}
    };

    int n = sizeof(inventario) / sizeof(inventario[0]);

    /* Mostrar el array original */
    printf("Inventario Original:\n");
    for (int i = 0; i < n; i++) {
        printf("%d Nro: %d | Nombre: %-15s | Precio: $%.2f\n",
            i + 1,
            inventario[i].nroProducto,
            inventario[i].nombre,
            inventario[i].precio);
    }

    /* Ordenamiento Burbuja por nroProducto (ascendente) */
    struct Producto temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (inventario[j].nroProducto > inventario[j + 1].nroProducto) {
                temp = inventario[j];
                inventario[j] = inventario[j + 1];
                inventario[j + 1] = temp;
            }
        }
    }

    /* Busqueda Binaria con valor fijo */
    int nroProductoBuscado = 58293012;
    int inicio = 0, fin = n - 1;
    struct Producto productoEncontrado = {0, "", 0.0};

    while (inicio <= fin) {

```

```
int medio = inicio + (fin - inicio) / 2;

if (inventario[medio].nroProducto == nroProductoBuscado) {
    productoEncontrado = inventario[medio];
    printf("\nProducto encontrado: %s - $%.2f\n",
           productoEncontrado.nombre,
           productoEncontrado.precio);
    break;
}

if (inventario[medio].nroProducto < nroProductoBuscado)
    inicio = medio + 1;
else
    fin = medio - 1;
}

/* Mostrar inventario ordenado */
printf("\nInventario ordenado por numero de producto:\n");
for (int i = 0; i < n; i++) {
    printf("%d Nro: %d | Nombre: %-15s | Precio: $%.2f\n",
           i + 1,
           inventario[i].nroProducto,
           inventario[i].nombre,
           inventario[i].precio);
}

return 0;
}
```

2. Inventario con Búsqueda Binaria Interactiva (ingreso por teclado)

Igual a la práctica anterior pero el número de producto lo ingresa el usuario con scanf. Una vez encontrado, imprime el nombre y precio del producto. Si no existe, informa al usuario.

```
#include <stdio.h>

/* Definir la estructura de datos */
struct Producto {
    int nroProducto;
    char nombre[20];
    float precio;
};

int main() {
    struct Producto inventario[] = {
        {58293012, "Monitor", 120.00},
        {19485726, "Mousepad", 10.00},
        {83746501, "Procesador", 200.00},
        {37482910, "Teclado", 25.50},
        {92837465, "Fuente ATX", 75.00},
        {48291037, "Webcam", 55.00},
        {10293847, "Silla Gamer", 250.00},
        {74659283, "Impresora", 85.00},
        {29384756, "Disco SSD", 95.00},
        {65748392, "Mouse", 15.75},
        {51928374, "Parlantes", 40.20},
        {84756291, "Cables HDMI", 8.50},
        {38475629, "Auriculares", 65.50},
        {91028374, "Gabinete", 60.00},
        {27384950, "Microfono", 30.00},
        {46573829, "Adaptador USB", 5.99},
        {15263748, "Memoria RAM", 50.00},
        {73849506, "Pendrive", 12.99},
        {62738495, "Placa Video", 350.00},
        {39485761, "Escritorio", 150.00}
    };

    int n = sizeof(inventario) / sizeof(inventario[0]);

    /* Mostrar el array original */
    printf("Inventario Original:\n");
    for (int i = 0; i < n; i++) {
```

```

        printf("%d) Nro: %d | Nombre: %-15s | Precio: $%.2f\n",
               i + 1,
               inventario[i].nroProducto,
               inventario[i].nombre,
               inventario[i].precio);
    }

    /* Ordenamiento Burbuja por nroProducto */
    struct Producto temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (inventario[j].nroProducto > inventario[j + 1].nroProducto) {
                temp = inventario[j];
                inventario[j] = inventario[j + 1];
                inventario[j + 1] = temp;
            }
        }
    }

    /* Búsqueda Binaria interactiva */
    int nroProductoBuscado;
    printf("\nIngrese el numero de producto a buscar: ");
    scanf("%d", &nroProductoBuscado);

    int inicio = 0, fin = n - 1;
    int encontrado = 0;
    struct Producto productoEncontrado;

    while (inicio <= fin) {
        int medio = inicio + (fin - inicio) / 2;
        if (inventario[medio].nroProducto == nroProductoBuscado) {
            productoEncontrado = inventario[medio];
            encontrado = 1;
            break;
        }
        if (inventario[medio].nroProducto < nroProductoBuscado)
            inicio = medio + 1;
        else
            fin = medio - 1;
    }

    if (encontrado) {

```

```
    printf("\nProducto encontrado:\n");
    printf("Nombre : %s\n", productoEncontrado.nombre);
    printf("Precio : $%.2f\n", productoEncontrado.precio);
} else {
    printf("\nProducto con numero %d no encontrado.\n", nroProductoBuscado);
}

return 0;
}
```

3. CRUD Completo de Estudiantes con Menú Interactivo

Sistema de gestión de estudiantes con struct (dni, nombre, apellido, carrera). Menú do-while con 5 funciones: listar, buscar (binaria por DNI), eliminar, agregar (reordena con burbuja) y modificar. Capacidad para 1000 registros.

```
#include <stdio.h>
#include <string.h>
#define MAX_ESTUDIANTES 1000

/* Definir la estructura de datos */
struct Estudiante {
    int dni;
    char nombre[20];
    char apellido[20];
    char carrera[20];
};

/* Prototipos de funciones */
struct Estudiante buscarEstudiante(struct Estudiante lista[], int n);
void listarEstudiantes(struct Estudiante lista[], int n);
void eliminarEstudiante(struct Estudiante lista[], int *n);
void agregarEstudiante(struct Estudiante lista[], int *n);
void modificarEstudiante(struct Estudiante lista[], int n);
void ordenarPorDni(struct Estudiante lista[], int n);

int main() {
    /* Datos iniciales de ejemplo */
    struct Estudiante lista[MAX_ESTUDIANTES] = {
        {389628, "Jose", "Garcia", "TUP"},
        {2716753, "Lopez", "Perez", "FOL"},
        {128781, "Carlos", "Romero", "Poli"},
        {18738, "Kike", "Fernandez", "UNNE"},
        {187382, "Jesus", "Molina", "Rino"},
        {182378, "Alex", "Torres", "Forn"}
    };

    int n = 6;

    /* Ordenar al inicio */
    ordenarPorDni(lista, n);

    int opcion;
    do {
```

```

printf("\n===== MENU ESTUDIANTES =====\n");
printf("1) Listar estudiantes\n");
printf("2) Buscar estudiante\n");
printf("3) Eliminar estudiante\n");
printf("4) Agregar estudiante\n");
printf("5) Modificar estudiante\n");
printf("6) Salir\n");
printf("Seleccione una opcion: ");
scanf("%d", &opcion);

switch (opcion) {
    case 1:
        printf("\nEstudiantes ordenados por DNI:\n");
        listarEstudiantes(lista, n);
        break;
    case 2: {
        struct Estudiante encontrado = buscarEstudiante(lista, n);
        if (encontrado.dni != 0) {
            printf("\nEstudiante encontrado:\n");
            printf("DNI : %d\n", encontrado.dni);
            printf("Nombre : %s\n", encontrado.nombre);
            printf("Apellido : %s\n", encontrado.apellido);
            printf("Carrera : %s\n", encontrado.carrera);
        } else {
            printf("\nEstudiante no encontrado.\n");
        }
        break;
    }
    case 3:
        eliminarEstudiante(lista, &n);
        break;
    case 4:
        agregarEstudiante(lista, &n);
        break;
    case 5:
        modificarEstudiante(lista, n);
        break;
    case 6:
        printf("\nSaliendo del programa...\n");
        break;
    default:
        printf("\nOpcion invalida. Intente nuevamente.\n");

```

```

    }
} while (opcion != 6);

return 0;
}

/* Ordenamiento Burbuja por DNI (ascendente) */
void ordenarPorDni(struct Estudiante lista[], int n) {
    struct Estudiante temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (lista[j].dni > lista[j + 1].dni) {
                temp = lista[j];
                lista[j] = lista[j + 1];
                lista[j + 1] = temp;
            }
        }
    }
}

/* Busqueda Binaria por DNI */
struct Estudiante buscarEstudiante(struct Estudiante lista[], int n) {
    int dniBuscado;
    printf("Ingrese el DNI a buscar: ");
    scanf("%d", &dniBuscado);

    int inicio = 0, fin = n - 1;
    while (inicio <= fin) {
        int medio = inicio + (fin - inicio) / 2;
        if (lista[medio].dni == dniBuscado)
            return lista[medio];
        if (lista[medio].dni < dniBuscado)
            inicio = medio + 1;
        else
            fin = medio - 1;
    }

    struct Estudiante vacio = {0, "", "", ""};
    return vacio;
}

void listarEstudiantes(struct Estudiante lista[], int n) {

```

```

printf("%-4s %-10s %-20s %-20s %-15s\n",
        "#", "DNI", "Nombre", "Apellido", "Carrera");
printf("-----\n");
for (int i = 0; i < n; i++) {
    printf("%-4d %-10d %-20s %-20s %-15s\n",
            i + 1,
            lista[i].dni,
            lista[i].nombre,
            lista[i].apellido,
            lista[i].carrera);
}
}

void eliminarEstudiante(struct Estudiante lista[], int *n) {
    int dniEliminar;
    printf("Ingrese el DNI del estudiante a eliminar: ");
    scanf("%d", &dniEliminar);

    int pos = -1;
    for (int i = 0; i < *n; i++) {
        if (lista[i].dni == dniEliminar) { pos = i; break; }
    }
    if (pos == -1) { printf("\nEstudiante no encontrado.\n"); return; }

    for (int j = pos; j < *n - 1; j++)
        lista[j] = lista[j + 1];
    (*n)--;
    printf("\nEstudiante eliminado correctamente.\n");
}

void agregarEstudiante(struct Estudiante lista[], int *n) {
    if (*n >= MAX_ESTUDIANTES) {
        printf("\nNo se pueden agregar mas estudiantes. Lista llena.\n");
        return;
    }
    struct Estudiante nuevo;
    printf("Ingrese el DNI : "); scanf("%d", &nuevo.dni);
    printf("Ingrese el nombre : "); scanf(" %19[^\n]", nuevo.nombre);
    printf("Ingrese el apellido : "); scanf(" %19[^\n]", nuevo.apellido);
    printf("Ingrese la carrera : "); scanf(" %19[^\n]", nuevo.carrera);

    lista[*n] = nuevo;
}

```

```

    (*n)++;
    ordenarPorDni(lista, *n);
    printf("\nEstudiante agregado y lista reordenada.\n");
}

void modificarEstudiante(struct Estudiante lista[], int n) {
    int dniModificar;
    printf("Ingrese el DNI del estudiante a modificar: ");
    scanf("%d", &dniModificar);

    int indice = -1;
    for (int i = 0; i < n; i++) {
        if (lista[i].dni == dniModificar) { indice = i; break; }
    }
    if (indice == -1) { printf("\nEstudiante no encontrado.\n"); return; }

    printf("Ingrese el nuevo DNI : "); scanf("%d", &lista[indice].dni);
    printf("Ingrese el nuevo nombre : "); scanf(" %19[^\n]", lista[indice].nombre);
    printf("Ingrese el nuevo apellido : "); scanf(" %19[^\n]",
lista[indice].apellido);
    printf("Ingrese la nueva carrera : "); scanf(" %19[^\n]", lista[indice].carrera);
    printf("\nEstudiante modificado correctamente.\n");
}

```

4. CRUD de Array de Enteros con Menú Interactivo

Sistema de gestión de un array de números enteros. Menú do-while con: listar, buscar (binaria), eliminar, agregar y modificar. El array se mantiene siempre ordenado de menor a mayor.

```
#include <stdio.h>

#define MAX 1000

/* Prototipos */
void ordenar(int arr[], int n);
void listar(int arr[], int n);
int buscar(int arr[], int n);
void eliminar(int arr[], int *n);
void agregar(int arr[], int *n);
void modificar(int arr[], int n);

int main() {
    int numeros[MAX] = {15, 42, 8, 97, 23, 61, 5, 38, 74, 19};
    int n = 10;

    /* Ordenar al inicio */
    ordenar(numeros, n);

    int opcion;
    do {
        printf("\n===== MENU - ARRAY DE ENTEROS =====\n");
        printf("1) Listar numeros\n");
        printf("2) Buscar numero\n");
        printf("3) Eliminar numero\n");
        printf("4) Agregar numero\n");
        printf("5) Modificar numero\n");
        printf("6) Salir\n");
        printf("Seleccione una opcion: ");
        scanf("%d", &opcion);

        switch (opcion) {
            case 1: listar(numeros, n); break;
            case 2: buscar(numeros, n); break;
            case 3: eliminar(numeros, &n); break;
            case 4: agregar(numeros, &n); break;
            case 5: modificar(numeros, n); break;
            case 6: printf("\nSaliendo...\n"); break;
            default: printf("\nOpcion invalida.\n");
        }
    } while (opcion != 6);
}
```

```

    }
} while (opcion != 6);

return 0;
}

/* Ordenamiento Burbuja ascendente */
void ordenar(int arr[], int n) {
    int temp;
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
}

/* Muestra todos los elementos con su indice */
void listar(int arr[], int n) {
    if (n == 0) { printf("El array esta vacio.\n"); return; }
    printf("\nNumeros en el array (%d elementos):\n", n);
    for (int i = 0; i < n; i++)
        printf(" [%d] %d\n", i + 1, arr[i]);
}

/* Busqueda Binaria - retorna el indice o -1 si no existe */
int buscar(int arr[], int n) {
    int valor;
    printf("Ingrese el numero a buscar: ");
    scanf("%d", &valor);

    int inicio = 0, fin = n - 1;
    while (inicio <= fin) {
        int medio = inicio + (fin - inicio) / 2;
        if (arr[medio] == valor) {
            printf("Numero %d encontrado en la posicion [%d].\n", valor, medio + 1);
            return medio;
        }
        if (arr[medio] < valor) inicio = medio + 1;
        else fin = medio - 1;
    }
}

```

```

    printf("Numero %d no encontrado.\n", valor);
    return -1;
}

/* Busca el valor y lo elimina desplazando el resto */
void eliminar(int arr[], int *n) {
    int valor;
    printf("Ingrese el numero a eliminar: ");
    scanf("%d", &valor);

    int pos = -1;
    for (int i = 0; i < *n; i++)
        if (arr[i] == valor) { pos = i; break; }

    if (pos == -1) { printf("Numero no encontrado.\n"); return; }

    for (int j = pos; j < *n - 1; j++)
        arr[j] = arr[j + 1];
    (*n)--;
    printf("Numero %d eliminado correctamente.\n", valor);
}

/* Agrega un numero y reordena el array */
void agregar(int arr[], int *n) {
    if (*n >= MAX) { printf("Array lleno.\n"); return; }
    int valor;
    printf("Ingrese el numero a agregar: ");
    scanf("%d", &valor);
    arr[(*n)++] = valor;
    ordenar(arr, *n);
    printf("Numero %d agregado y array reordenado.\n", valor);
}

/* Busca un valor existente y lo reemplaza por uno nuevo */
void modificar(int arr[], int n) {
    int viejo, nuevo;
    printf("Ingrese el numero a modificar: ");
    scanf("%d", &viejo);

    int indice = -1;
    for (int i = 0; i < n; i++)
        if (arr[i] == viejo) { indice = i; break; }

```

```
if (indice == -1) { printf("Numero no encontrado.\n"); return; }  
  
printf("Ingrese el nuevo valor: ");  
scanf("%d", &nuevo);  
arr[indice] = nuevo;  
ordenar(arr, n);  
printf("Numero modificado: %d -> %d\n", viejo, nuevo);  
}
```

5. CRUD de Array de Strings con Menú Interactivo

Igual al anterior pero para cadenas de caracteres. Usa strcmp para comparar en el ordenamiento burbuja y en la búsqueda binaria alfabética. Permite listar, buscar, eliminar, agregar y modificar palabras.

```
#include <stdio.h>
#include <string.h>
#define MAX 1000
#define TAM 50

/* Prototipos */
void ordenar(char arr[][TAM], int n);
void listar(char arr[][TAM], int n);
void buscar(char arr[][TAM], int n);
void eliminar(char arr[][TAM], int *n);
void agregar(char arr[][TAM], int *n);
void modificar(char arr[][TAM], int n);

int main() {
    char palabras[MAX][TAM] = {
        "manzana", "naranja", "banana", "uva", "pera",
        "limon", "mango", "fresa", "kiwi", "melon"
    };
    int n = 10;

    /* Ordenar alfabeticamente al inicio */
    ordenar(palabras, n);

    int opcion;
    do {
        printf("\n===== MENU - ARRAY DE STRINGS =====\n");
        printf("1) Listar palabras\n");
        printf("2) Buscar palabra\n");
        printf("3) Eliminar palabra\n");
        printf("4) Agregar palabra\n");
        printf("5) Modificar palabra\n");
        printf("6) Salir\n");
        printf("Seleccione una opcion: ");
        scanf("%d", &opcion);

        switch (opcion) {
            case 1: listar(palabras, n); break;
            case 2: buscar(palabras, n); break;
```

```

        case 3: eliminar(palabras, &n); break;
        case 4: agregar(palabras, &n); break;
        case 5: modificar(palabras, n); break;
        case 6: printf("\nSaliendo...\n"); break;
        default: printf("\nOpcion invalida.\n");
    }
} while (opcion != 6);

return 0;
}

```

```

/* Ordenamiento Burbuja alfabetico usando strcmp */

```

```

void ordenar(char arr[][TAM], int n) {
    char temp[TAM];
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)
            if (strcmp(arr[j], arr[j + 1]) > 0) {
                strcpy(temp, arr[j]);
                strcpy(arr[j], arr[j + 1]);
                strcpy(arr[j + 1], temp);
            }
}

```

```

/* Muestra todas las palabras con su indice */

```

```

void listar(char arr[][TAM], int n) {
    if (n == 0) { printf("El array esta vacio.\n"); return; }
    printf("\nPalabras en el array (%d elementos):\n", n);
    for (int i = 0; i < n; i++)
        printf(" [%d] %s\n", i + 1, arr[i]);
}

```

```

/* Busqueda Binaria alfabetica con strcmp */

```

```

void buscar(char arr[][TAM], int n) {
    char valor[TAM];
    printf("Ingrese la palabra a buscar: ");
    scanf(" %49[^\n]", valor);

    int inicio = 0, fin = n - 1;
    while (inicio <= fin) {
        int medio = inicio + (fin - inicio) / 2;
        int cmp = strcmp(arr[medio], valor);
        if (cmp == 0) {

```

```

        printf("Palabra \"%s\" encontrada en posicion [%d].\n", valor, medio +
1);
        return;
    }
    if (cmp < 0) inicio = medio + 1;
    else fin = medio - 1;
}
printf("Palabra \"%s\" no encontrada.\n", valor);
}

/* Busca la palabra y la elimina desplazando el resto */
void eliminar(char arr[][TAM], int *n) {
    char valor[TAM];
    printf("Ingrese la palabra a eliminar: ");
    scanf(" %49[^\n]", valor);

    int pos = -1;
    for (int i = 0; i < *n; i++)
        if (strcmp(arr[i], valor) == 0) { pos = i; break; }

    if (pos == -1) { printf("Palabra no encontrada.\n"); return; }

    for (int j = pos; j < *n - 1; j++)
        strcpy(arr[j], arr[j + 1]);
    (*n)--;
    printf("Palabra \"%s\" eliminada correctamente.\n", valor);
}

/* Agrega una nueva palabra y reordena el array */
void agregar(char arr[][TAM], int *n) {
    if (*n >= MAX) { printf("Array lleno.\n"); return; }
    char valor[TAM];
    printf("Ingrese la nueva palabra: ");
    scanf(" %49[^\n]", valor);
    strcpy(arr[(*n)++], valor);
    ordenar(arr, *n);
    printf("Palabra \"%s\" agregada y array reordenado.\n", valor);
}

/* Busca una palabra existente y la reemplaza por otra */
void modificar(char arr[][TAM], int n) {
    char vieja[TAM], nueva[TAM];

```

```
printf("Ingrese la palabra a modificar: ");
scanf(" %49[^\n]", vieja);

int indice = -1;
for (int i = 0; i < n; i++)
    if (strcmp(arr[i], vieja) == 0) { indice = i; break; }

if (indice == -1) { printf("Palabra no encontrada.\n"); return; }

printf("Ingrese la nueva palabra: ");
scanf(" %49[^\n]", nueva);
strcpy(arr[indice], nueva);
ordenar(arr, n);
printf("Palabra modificada: \"%s\" -> \"%s\"\n", vieja, nueva);
}
```